

La Shell GNU/Linux

La **Shell** è un programma che funziona **dentro un terminale e interpreta** tutto ciò che viene scritto. Se trova un **comando builtin**, lo esegue direttamente; altrimenti cerca un **programma esterno** nel percorso definito dalla variabile **PATH**.

Per avere informazioni sui comandi builtin si può usare il comando `help`.

Invece, per avere informazioni sui programmi e su altri elementi del sistema (come file di configurazione), si utilizza il comando `man`.

Per sapere come usare `man`, basta digitare `man man`.

Per visualizzare tutte le variabili di ambiente basta digitare `env`. Per visualizzare solo **PATH**, digitare `env | grep PATH`.

La shell in uso è memorizzata nella variabile di ambiente **SHELL**, cosa digitare per visualizzarla?

Noi prenderemo come riferimento la Shell GNU Bash `/usr/bin/bash`, che è la più diffusa, ma molte altre le somigliano.

NOTA: leggendo queste righe introduttive focalizzati sulla **differenza tra comando builtin e programma esterno**.

Esempio di help e di man

```
$ help cd
cd: cd [-L|[-P [-e]] [-@]] [dir]
    Cambia la directory corrente.

$ man ls
LS(1)                                Manuale utente Linux                                LS(1)
NAME
    ls - elenca il contenuto di una directory
```

`cd` è un **comando builtin** (incluso nella Bash)

`ls` è un **programma esterno**

Per distinguere se un comando è builtin o programma eseguibile del sistema si può usare `which`.

```
root@lab-debian:~# which cd
root@lab-debian:~# which ls
/usr/bin/ls
root@lab-debian:~# which bash
/usr/bin/bash
```

Uno sguardo su qualcosa di più avanzato

Inizialmente non capirai, né saprai usare tutto quello che c'è scritto qui. Quando saprai descrivere con sicurezza ogni passaggio nella parte introduttiva, sarai già a buon punto e potrai leggere con più facilità gli esempi che seguono, per poi proseguire autonomamente.

Esempi di redirectione, di pipe e di qualche comando base:

se si volessero salvare le variabili di ambiente in un file si potrebbe fare `env > ~/variabili.txt` che sarebbe uguale a scrivere `env > $HOME/variabili.txt`. Se invece vuoi solo le prime 10 righe scriverai `env | head > ~/variabili.txt`.

Se invece volessi le email degli autori dei commit su un repository Git che conosciamo, basta posizionarsi nel workspace e digitare

```
git log | grep "Author:" | cut -d"<" -f2 | cut -d">" -f1 | sort | uniq > email_autori.txt
```

Il Terminale

```

feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux$
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux$
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux$
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux$ cd
feroda@paco: ~$ pwd
/home/feroda
feroda@paco: ~$ cd -
/home/feroda/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux$ pwd
/home/feroda/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux$ ls
intro.md
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis/02_shell_GNULinux$ cd ..
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis$
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis$ ls -lrt
totale 32
-rwxrwxr-- 1 feroda feroda 138 ott 3 09:03 create_structure.sh
drwxrwxr-x 3 feroda feroda 4096 ott 3 09:16 assets
drwxrwxr-x 2 feroda feroda 4096 ott 4 18:35 00_requisiti-del-corso
-rwxrwxr-- 1 feroda feroda 224 ott 5 09:26 md2pdf.sh
drwxrwxr-x 2 feroda feroda 4096 ott 15 07:33 01_vcs
drwxrwxr-x 2 feroda feroda 4096 ott 31 17:59 99_progetti
-rw-rw-r-- 1 feroda feroda 1372 ott 31 18:01 README.md
drwxrwxr-x 2 feroda feroda 4096 ott 31 19:15 02_shell_GNULinux
feroda@paco: ~/Scuola/2025 - IIS Merloni Miliani TPSIT/src/tpsit-4o-anno-itis$

```

Quindi la SHELL viene eseguita dentro questa finestra, vediamo alcune caratteristiche. Innanzitutto la riga che vedete ripetuta più volte è il **PROMPT**, ossia è la riga che ci indica che la SHELL è pronta a ricevere comandi. Nell'ultima riga in fondo si vede evidenziato anche il cursore dove verranno inseriti i caratteri.

La riga in questo caso è composta da:

- **username:** in verde, l'utente che attualmente ha eseguito la SHELL
- **@ :** separatore tra utente e host
- **hostname:** il nome del sistema su cui è installato il sistema operativo
- **:** separatore tra host e directory
- **directory:** la directory in cui l'utente si trova, sinonimo di "."
- **\$:** simbolo che ci indica che questo utente non è privilegiato. La **#** invece avrebbe indicato che l'utente è privilegiato (utente root o superutente. L'equivalente di Administrator nei sistemi Windows)

Tra un invio ed un altro vedete l'esecuzione dei comandi `cd`, `pwd` e `ls`. Sono programmi o comandi builtin della SHELL? In quale directory risiedono quegli eseguibili? Trovare per esercizio

Accedere ad un sistema remoto via ssh - Secure Shell

Possiamo lavorare con la bash in vari modi:

- con un sistema GNU/Linux
- possiamo anche usare la Git Bash, che però è limitata
- o possiamo usare WSL, il sottosistema Linux per Windows

Il modo ufficiale per lavorare con la Bash nel corso TPSIT è di connettersi ad un sistema remoto, il server Debian GNU/Linux del Dipartimento di Informatica della nostra scuola.

`ssh utente@<IP del server Debian>` e password concordata finché non avrete un utente specifico. [SSH](#) è l'acronimo di Secure Shell che può indicare sia un protocollo di sicurezza, sia un software. La sua implementazione libera è OpenSSH. È molto potente e **universalmente** usato.

Compagno del software SSH è SCP - Secure Copy, che fa parte sempre dello stesso gruppo di programmi e serve per copiare files da locale a remoto e da remoto a locale.

`scp utente@<IP del server Debian>:/etc/hosts .` trasferisce il file remoto `/etc/hosts` nella directory corrente del sistema locale, invece

`scp pippo.pdf utente@<IP del server Debian>:~` trasferisce il file `pippo.pdf` che risiede nella directory corrente del sistema locale sul server Debian, nella directory home (identificata da `~` o da `$HOME`).

Opzione molto usata in `ssh` è `-p` per decidere la porta su cui connettersi al servizio ssh (sshd). Opzioni molto usate in `scp` sono `-P` per decidere la porta su cui connettersi al servizio (sshd) e `-r` per una copia ricorsiva, ossia copiare le directory.

FHS – Filesystem Hierarchy Standard / La struttura delle directory nei sistemi Unix-like

Path	Descrizione
<code>/</code>	Root del filesystem, punto di partenza di tutto.
<code>/bin</code>	Programmi essenziali per tutti gli utenti (ls, cp, mv...).
<code>/sbin</code>	Programmi essenziali per l'amministrazione (mount, sysctl...).
<code>/etc</code>	File di configurazione del sistema.
<code>/home</code>	Home degli utenti normali.
<code>/root</code>	Home dell'utente root.
<code>/lib</code> , <code>/lib64</code>	Librerie necessarie ai programmi in <code>/bin</code> e <code>/sbin</code> .
<code>/usr</code>	Programmi e file non essenziali ma importanti.

Path	Descrizione
/usr/bin	Programmi utente non vitali (vim, python, gcc...).
/usr/sbin	Programmi amministrativi non essenziali.
/usr/lib	Librerie per i programmi in /usr/bin e /usr/sbin.
/var	Dati variabili: log, spool, cache, database interni.
/var/log	Log di sistema.
/var/lib	Dati persistenti dei servizi (dpkg, docker...).
/var/cache	Cache dei programmi.
/tmp	File temporanei, cancellabili al riavvio.
/run	Stato runtime del sistema (volatile).
/dev	Device file: dischi, terminali, pseudo-device.
/proc	Informazioni del kernel come pseudo-filesystem.
/sys	Interfaccia verso kernel e driver.
/mnt	Mount temporanei gestiti dall'amministratore.
/media	Mount automatici (USB, CD...).
/opt	Software opzionale di terze parti.
/srv	Dati forniti da servizi (web, FTP...).



Comandi della SHELL GNU Bash e programmi GNU/Linux



Navigazione e gestione file

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
ls	programma	Elenca file e directory	-l formato lungo, -a file nascosti, -lrt ordina per data inversa	ls -la /etc

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
cd	builtin	Cambia directory	cd - torna alla precedente, cd .. padre	cd /var/log
pwd	builtin	Mostra la directory corrente	—	pwd
cp	programma	Copia file o directory	-r ricorsivo, -a preserva attributi	cp -a /etc /backup
mv	programma	Sposta o rinomina	-i chiede conferma	mv prova.txt documenti/
rm	programma	Rimuove file o directory	-r ricorsivo, -f forza	rm -rf old_logs/
mkdir	programma	Crea directory	-p crea anche genitori mancanti	mkdir -p /data/backup/oggi
find	programma	Cerca file (avanzato)	-name per nome, -type per tipo	find /etc -name "*.conf"
chmod	programma	Modifica i permessi di un file	-R ricorsivo	chmod -R 755 /srv/www
chown	programma	Cambia proprietario di file	-R ricorsivo	chown root:root /etc/passwd
tar	programma	Gestisce archivi anche compressi	cvfz , xvfz	tar cvfz backup.tar.gz /etc
gzip	programma	Comprimi/decomprime file	-d decomprime	gzip file.txt

Visualizzazione e testo

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
cat	programma	Mostra file di testo	—	cat /etc/hostname

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
less , more	programma	Scorrono l'output a pagine	-N numeri di riga	cat /etc/passwd less -N
grep	programma	Cerca testo nei file	-r ricorsivo, -i case-insensitive	grep -ri "error" /var/log
head , tail	programma	Mostrano inizio/fine file	-n numero righe, -f segue in tempo reale	tail -f /var/log/syslog
sort	programma	Ordina righe	-r inverso, -n numerico	sort -nr dati.txt
wc	programma	Conta righe, parole, byte	-l , -w , -c	wc -l /etc/passwd
cut	programma	Estrae colonne da testo	-d delimitatore, -f campo	cut -d: -f1 /etc/passwd
tree	programma	Mostra struttura ad albero	-L profondità	tree -L 2 /etc
sed	programma	Modifica testo per righe	-e script inline, -i in place	sed -i 's/foo/bar/g' file.txt
awk	programma	Elabora testo per campi	- F delimitatore, {print} azione	awk -F: '{print \$1}' /etc/passwd
vim	programma	Editor di testo potente	—	vim file.txt

Sistema e processi

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
ps	programma	Mostra processi	aux tutti i processi	ps aux grep nginx

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
top	programma	Mostra processi live	—	top
htop	programma	Versione colorata e interattiva	—	htop
kill, killall	builtin / programma	Termina processi	-9 forza	kill -9 1234
free	programma	Mostra uso memoria	-h formato leggibile	free -h
df	programma	Spazio su disco	-h formato leggibile	df -h
du	programma	Spazio usato da directory	-h, -s sommario	du -sh /var/log
uptime	programma	Mostra tempo attività	—	uptime
shutdown	programma	Spegne o riavvia	-r reboot, -h halt	shutdown -r now
hostnamectl	programma	Mostra/modifica hostname	—	hostnamectl set-hostname lab
timedatectl	programma	Gestisce data/ora	set-time, set-timezone	timedatectl set-timezone Europe/Rome
systemctl	programma	Gestisce servizi systemd	start, status, enable	systemctl status ssh
journalctl	programma	Log systemd	-xe errori, -f segui log	journalctl -u ssh -f



Ambiente utente

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
history	builtin	Mostra cronologia comandi	—	history tail
sudo	programma	Esegue comandi come root	-i shell root	sudo apt update

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
env	builtin	Mostra variabili d'ambiente	—	env grep PATH
echo	builtin	Mostra testo o variabili	—	echo \$SHELL
whoami / id	programma	Mostra utente corrente e UID	—	id
w	programma	Mostra utenti loggati	—	w
groups	programma	Mostra gruppi utente	—	groups
passwd	programma	Cambia password	—	passwd

Aiuto e manuali

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
help	builtin	Info sui comandi Bash	—	help cd
man	programma	Manuale dei comandi	-k ricerca parola chiave	man -k network
whatis	programma	Breve descrizione comando	—	whatis grep
apropos	programma	Cerca per parola chiave	—	apropos copy

Rete

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
ping	programma	Test connessione	-c numero pacchetti	ping -c 4 google.com
ip	programma	Mostra e configura rete	a, r, l	ip addr list, ip link show
ss	programma	Porte e connessioni	-tuln TCP/UDP	ss -tuln

Comando	Tipo	Descrizione	Opzioni utili	Esempio di uso
netstat	programma	Porte e connessioni. Equivalente a ss, ma più vecchio e diffuso	-pltn TCP in LISTEN con nome processi	netstat -pltn
wget	programma	Scarica file da web	-c continua download	wget -c https://example.com/file
curl	programma	Richiesta HTTP	-O salva file	curl -O https://example.com
ssh	programma	Connessione remota sicura	-p porta	ssh -p 2222 user@server
scp	programma	Copia sicura tra host	-r ricorsivo	scp -r cartella user@server:/tmp
nc	programma	NetCat. Socket client/server TCP/UDP	-l listen	avvia server nc -l 0.0.0.0 1024 e il client nc 127.0.0.1 1024
telnet	programma	Socket client TCP (superato da nc)		client telnet 127.0.0.1 1024
tcpdump	programma	Cattura il traffico di rete	-n -i eth0	tcpdump -n -i eth0 -s0 -w/tmp/packets.pcap port 1024
wireshark	programma	Analisi del traffico di rete		wireshark /tmp/packets.pcap
nmap	programma	Scansione di host e reti	-O riconosce OS, -p 21-1024	nmap 192.168.1.1/24 -O -ss

Redirezione in bash: | > 2> < &> >>

La Bash permette di **reindirizzare** l'input e l'output dei comandi usando operatori speciali. Questi servono per collegare comandi tra loro, salvare risultati in file e separare output normale ed errori.

> – redirezione dell'output standard (stdout)

Scrivo l'output del comando in un file, **sovrascrivendolo**.

```
ls > elenco.txt
```

>> – aggiunta (append)

Aggiunge l'output alla fine di un file esistente.

```
echo "nuova riga" >> log.txt
```

2> – redirectione dell'errore standard (stderr)

Salva SOLO gli errori in un file.

```
grep foo /non/esiste 2> errori.log
```

&> – redirectione combinata (stdout + stderr)

Manda *sia* l'output normale che gli errori nello stesso file.

```
comando &> tutto.log
```

< – redirectione dell'input (stdin)

Usa un file come input al posto della tastiera.

```
sort < nomi.txt
```

| – pipe (collega comandi)

Invia l'output del primo comando come **input** del secondo.

```
cat /etc/passwd | grep root
```

In sintesi:

esistono 3 files descriptor:

- 0. stdin = standard input
- 1. stdout = standard output
- 2. stderr = standard error

Operatore	Significato
>	stdout → file (sovrascrivi)
>>	stdout → file (append)
2>	stderr → file
&>	stdout + stderr → file

Operatore	Significato
<	file → stdin
,	,



Mini guida su sed, awk e vim

sed — Stream Editor

Serve per modificare testo riga per riga:

```
sed 's/vecchio/nuovo/g' file.txt      # sostituzione in output
sed -i 's/foo/bar/g' file.txt         # sostituzione diretta nel file
sed -n '/error/p' file.log            # stampa solo le righe con "error"
```

awk — Elaboratore di campi

Serve per estrarre o calcolare campi di un file strutturato:

```
awk -F: '{print $1}' /etc/passwd      # mostra solo la prima colonna
awk '{sum += $2} END {print sum}' dati.txt # somma la seconda colonna
awk '$3 > 1000 {print $1, $3}' file.txt  # filtra righe con campo 3 > 1000
```

vim — Editor di testo

Un editor potente, con due modalità principali:

- **Normale** (movimento/comandi)
- **Inserimento** (scrittura testo)

e una secondaria

- **Visuale** (selezione)

Comandi base

```
i          → entra in modalità inserimento
<ESC>      → torna alla modalità normale
:w         → salva
:q         → esce
:wq        → salva ed esce
:q!       → esce senza salvare
/TESTO    → cerca testo (in avanti)
?TESTO    → cerca testo (all'indietro)
```

```
n      → vai alla successiva occorrenza
dd     → cancella riga
yy     → copia riga
p      → incolla
.      → ripeti l'ultima operazione di inserimento
```

Comandi intermedi

```
h j k l → muovi il cursore a sinistra, giù, su, destra
o       → aggiungi una riga dopo ed entra in modalità inserimento
O       → aggiungi una riga prima ed entra in modalità inserimento
CTRL+p  → autocompleta la parola cercando indietro
CTRL+n  → autocompleta la parola cercando in avanti
CTRL+x+f → autocompleta la parola con i nomi dei files
w       → muove il cursore in avanti di una parola
b       → muove il cursore indietro di una parola
0       → muove il cursore ad inizio riga
$       → muove il cursore a fine riga
v       → seleziona in modalità Visuale
SHIFT+v → seleziona in modalità Visuale per righe
CTRL+v  → seleziona in modalità Visuale per blocchi
```

Comandi avanzati e potenti

```
:s/xxx/yyy/g      → sostituisce tutte le occorrenze di xxx con yyy nella riga corrente
:%s/xxx/yyy/g     → sostituisce in tutto il file, grazie al %
:%s/xxx/yyy/gc    → come sopra, ma con conferma, grazie alla c
:1,10s/xxx/yyy/gc → sostituisce tra la riga 1 e 10, con conferma, grazie a 1,10 come p

qa               → inizia a registrare una macro nel registro 'a'
q               → termina la registrazione
@a              → esegue la macro
10@a            → la esegue 10 volte
```